EX15b - Practica NumPy - Shape, slicing y broadcasting

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex15b_Numpy_shapes_and_operations_practice.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Practicar la geometria de arrays y las operaciones NumPy basicas antes de aplicarlas a datasets mayores.

Variables / elementos del problema

Nombre	Significado
vector_a	array 1D con 3 elementos
matrix_a	array 2D de shape (2,3)

Ecuacion o logica en papel

```
shape / ndim / size / dtype
broadcasting
reshape / transpose
vstack / hstack
* vs @
```

Comprobacion manual

matrix_a tiene shape (2,3), ndim=2 y size=6. vector_a tiene shape (3,), ndim=1 y size=3.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- np.array
- shape
- slicing basico

Nuevo en este ejercicio:

- ndim
- size
- dtype
- slicing 2D
- operaciones elemento a elemento
- broadcasting
- reshape
- transpose
- vstack/hstack
- axis
- argmax
- dot/cross
- @

3. Conceptos Python nuevos

1. ndim

Sintaxis: `A.ndim`

Cuenta ejes estructurales. vector_a[i] usa un indice; matrix_a[i,j] usa dos.

2. Broadcasting

Sintaxis: `matrix_a + offset`

NumPy compara shapes desde la derecha. (2,3) y (3,) son compatibles porque el ultimo 3 coincide.

3. reshape

Sintaxis: `vector_a.reshape(1,3)`

Cambia la estructura sin cambiar los tres datos.

4. axis

Sintaxis: `np.max(matrix_b, axis=0)`

axis=0 reduce filas y deja un resultado por columna; axis=1 reduce columnas y deja uno por fila.

5. @

Sintaxis: `A @ v`

@ realiza algebra matricial; * multiplica elemento a elemento.

4. Codigo paso a paso

Paso 1. Inspeccionar geometria

```
vector_a.shape
vector_a.ndim
vector_a.size
vector_a.dtype
```

Antes de operar conviene saber forma, numero de ejes, numero de elementos y tipo.

Paso 2. Indexar

```
matrix_a[0, 0]
matrix_a[0, :]
matrix_a[:, 1]
matrix_a[:, 1:]
```

Dos indices significan fila y columna. : representa todos o un rango.

Paso 3. Elemento a elemento

```
vector_a + vector_b
vector_a * vector_b
vector_a**2
```

NumPy aplica cada operador a las posiciones correspondientes.

Paso 4. Broadcasting

```
offset = np.array([100.0, 200.0, 300.0])
result = matrix_a + offset
```

El vector de shape (3,) se suma a cada fila de la matriz (2,3).

Paso 5. reshape y T

```
row = vector_a.reshape(1,3)
column = vector_a.reshape(3,1)
matrix_a_T = matrix_a.T
```

Mismos datos, estructuras distintas. T intercambia filas y columnas.

Paso 6. Apilar

```
np.vstack((matrix_a, new_row))
```

```
np.hstack((matrix_a, new_column))
```

vstack necesita columnas compatibles; hstack filas compatibles.

Paso 7. Reducir por ejes

```
np.max(matrix_b, axis=0)
np.max(matrix_b, axis=1)
```

El eje indicado desaparece en el resultado de la reduccion.

Paso 8. Algebra vectorial

```
vector_a @ vector_b
np.cross(vector_a, vector_b)
matrix_a @ vector_b
```

El ejercicio compara producto escalar, vectorial y matriz-vector.

5. Flujo de ejecucion

```
create arrays
 -> inspect geometry
 -> index/slice
 -> element-wise ops
 -> broadcasting
 -> reshape/stack
 -> axis reductions
 -> algebra
```

6. Resultado / comprobacion esperada

Entre otros resultados: vector_a@vector_b=32, cross=[-3,6,-3], matrix_a@vector_b=[32,77].

7. Errores tipicos al empezar

- Confundir ndim con dimension fisica.
- Pensar que (3,) y (1,3) son iguales.
- Creer que * es producto matricial.
- No comprobar shapes antes de broadcasting.
- Interpretar axis como eje fisico x/y/z.

8. Si solo recuerdas cinco cosas

- shape da longitudes por eje.
- ndim cuenta ejes estructurales.
- Broadcasting compara shapes desde la derecha.
- reshape cambia forma, no datos.
- * es elemento a elemento y @ es algebra matricial.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.